

FINAL YEAR PROJECT

CMS — News Portal Content Management System

External Viva Preparation Guide

PHP • MySQL • HTML/CSS • XAMPP (Apache)

Read this fully before Monday. It explains the whole project in simple words: what it is, how it works, every file, the database, security, expected viva questions with answers, and a demo checklist.

1. What Is This Project? (Simple Explanation)

Our project is a **news website with a built-in management system** — a mini version of sites like a newspaper portal or a blogging platform. It is called a **CMS (Content Management System)** because all the content (articles, comments, users) is created, approved and managed through the website itself — nobody has to touch the code to publish a news article.

Three kinds of people use it:

- **Visitors (guests)** — can read all published news articles, open a full article, and see its comments. They can also register or log in.
- **Registered users** — can additionally **write their own news articles** and **comment** on articles. Their articles are NOT published immediately — they wait for admin approval. They also get a personal dashboard (profile, my posts, my comments).
- **Admins** — get a separate admin dashboard with live statistics. They approve or reject pending articles, and manage all posts, users and comments on the site.

One-line summary to memorize: “It is a role-based news CMS built in PHP and MySQL where users submit articles, admins moderate and publish them, and visitors read them — with secure login using hashed passwords and sessions.”

2. Technology Stack

Technology	Where it is used	Why we chose it
PHP	All server-side logic: routing, login, database queries, form handling	Free, easy to learn, runs everywhere, made for web apps
MySQL (via mysqli)	Database “cms” with 3 tables: users, posts, comments	Reliable relational database; mysqli is PHP’s built-in MySQL driver
HTML5 + CSS3	All page design. Custom CSS (no Bootstrap), CSS variables for one consistent theme, plus a shared assets/beautify.css that polishes every page from one file. Fonts (Inter, Playfair Display) are self-hosted so the site looks identical offline.	Full design control; one place to theme the whole site; no internet dependency
JavaScript (small)	Alert boxes / redirects after form submissions, the newsletter acknowledgement, and subtle scroll-in animations (assets/beautify.js)	Simple interactivity and a modern feel without heavy libraries
Apache + XAMPP	Local web server that runs the PHP files	Standard, beginner-friendly local development package
.htaccess	URL rewriting (clean URLs)	Lets “/about.html” style links internally load index.php?page=about
PHP Sessions	Keeping users logged in across pages; storing id, username, role	Server-side and safer than storing login info in the browser

3. How To Run The Project (Demo Setup)

- 1. Open the **XAMPP Control Panel** and start **Apache** and **MySQL**.
- 2. The database is named **cms** (visible in phpMyAdmin) with tables **users**, **posts**, **comments**.
- 3. Open the browser at: **http://localhost:8080/de6/index.php** (our Apache runs on port 8080).
- 4. The site is pre-loaded with **45 realistic articles across 12 categories** (41 published, 4 pending for the approval demo) plus sample comments. Test members log in with password **123**; there is one **admin** account.
- 5. **Delete the development-only files before final submission** — the `seed_*.php` scripts, `assets/dev/reseed_articles.php`, and the `backup_pre_beautify/` folder are helpers, not part of the product.

4. Architecture — How The Whole Site Is Wired

4.1 The Master Router (Front Controller pattern)

The most important file is **index.php**. Every request goes through it — this design is called the **Front Controller pattern**. It does four jobs, in order:

- **Starts the session** once, so every page knows who is logged in.
- **Connects to the MySQL database** once (host: localhost, db: cms) and shares the connection variable **\$conn** with whichever page loads.
- **Reads the URL parameter ?page=...** (for example `index.php?page=login`) and uses a **switch-case** to load (require) the matching PHP file. If no page is given, it loads home.
- **Closes the database connection** after the page finishes.

Request flow (say it like this in the viva):

```
Browser requests index.php?page=single_post&id=5
1) index.php starts session + connects to DB
2) switch-case sees page=single_post, so it requires single_post.php
3) single_post.php queries MySQL for post id 5 and its comments
4) PHP mixes the data into HTML and sends the finished page back to the browser
```

4.2 Clean URLs (.htaccess)

The `.htaccess` file turns on Apache's **RewriteEngine**. Any link ending in `.html` (e.g. `about.html`) is silently redirected to `index.php?page=about`, and the site root `/` loads the home page. Real files like images are still served normally. Benefit: shorter, cleaner URLs and the internal file structure stays hidden.

4.3 Defensive connection fallback

Each page also contains a small safety block: `if (!isset($conn)) { connect again }`. So even if someone opens a page file directly (skipping the router), the page can still create its own database connection instead of crashing. Mention this if asked “what happens if a file is opened directly?”

5. Database Design (database name: cms)

Three tables, connected by foreign keys. One user can write many posts; one post can have many comments; every comment belongs to one user and one post.

Table	Main columns	Purpose
users	id (PK), username (unique), firstname, lastname, email (unique), password (hashed, never plain text), role (ENUM: 'reader', 'content_creator', 'admin', 'superadmin'), status ('active'/'pending'), created_at	Every account on the site. The role column decides what the person is allowed to do. A new sign-up defaults to 'reader'.
posts	id (PK), user_id (FK to users.id), title, content, image_path (uploaded file path OR an image URL), category, status ('pending' or 'published'), created_at	All news articles. status is the heart of the approval workflow: pending = waiting for admin, published = visible to everyone.
comments	id (PK), post_id (FK to posts.id), user_id (FK to users.id), comment text, created_at	Reader comments under each article.

JOINS used: when showing posts we join posts with users (JOIN users u ON p.user_id = u.id) to display the author's name next to each article; comments are joined the same way to show the commenter's username. This is an INNER JOIN — only rows that match on both sides are returned.

6. User Roles and Permissions

In the database the member-level roles are **reader** and **content_creator**; the administrator roles are **admin** and **superadmin**. The code only ever checks “admin/superadmin vs everyone else”, so any logged-in member (reader or content_creator) can write articles and comment. In the table below, “Member” means any logged-in non-admin account.

Ability	Guest	Member	Admin / Superadmin
Read published articles + comments	Yes	Yes	Yes
Register / login	Yes	—	—
Write an article (goes to 'pending')	No	Yes	Yes
Comment on articles	No	Yes	Yes
Personal dashboard (profile, my posts, my comments)	No	Yes	Yes
Admin dashboard with site statistics	No	No	Yes
Approve / manage ALL posts	No	No	Yes
Manage users (edit name, email, role)	No	No	Yes
Manage all comments	No	No	Yes

Every protected page begins with a security check. User pages check `$_SESSION['user_id']`; admin pages additionally check `$_SESSION['role']` is admin/superadmin, otherwise the visitor is thrown out (“Access Denied” or redirect to home). So typing an admin URL directly does NOT work without an admin login — this is **role-based access control**.

7. Every File Explained (Page Map)

File	Who can open	What it does
index.php	Everyone	Master router: session + DB connection + switch-case that loads the right page. Also handles ?page=logout (destroys the session).
.htaccess	(server)	Clean URL rewriting into index.php.
home.php	Everyone	Homepage: hero banner with LIVE stats (articles / members / categories), working category filter pills (built from real categories), latest published articles (first one featured), a quick login widget, Trending Now and a newsletter box. The sidebar login form is processed at the top of this file.
about.php	Everyone	About Us static page.
register.php	Guests	Registration form. Validates all fields, matching passwords (min 6 chars), valid email; checks duplicates with a prepared statement; hashes the password; inserts the new user.
register_action.php	Guests	Alternate standalone registration handler that responds with JavaScript alert boxes.
login.php	Guests	Full login page. Verifies password with password_verify(); saves id/username/role in the session; admins are redirected to the dashboard, users to home. Also supports "smart redirect" back to the article you were reading.
single_post.php	Everyone	Full article view: title, author, date, image, content, all comments (newest first) and a comment form for logged-in users. Also remembers this URL so login can bring you back here.
create_post.php	Logged-in users	Write Article page: title, content, category, and an image either uploaded from the computer (saved into uploads/ with a timestamped name) or given as a link. Saved with status 'pending'.
detail.php	Logged-in users	My Profile dashboard: personal info plus live counts of my posts and my comments.
my_posts.php	Logged-in users	List of my own articles with their status (pending/published).
my_comments.php	Logged-in users	List of all comments I have written.
admin.php	Admins only	Admin dashboard: total posts, total users, published count, pending count.
post.php	Admins only	Manage Posts: table of ALL posts (any status) with author names, for approval/management.
list.php	Admins only	Manage Users: table of all registered users.
edit_user.php	Admins only	Edit a user's name, email and role. Safety rule: an admin cannot change their OWN role here.

File	Who can open	What it does
comments.php	Admins only	Manage Comments: moderate all comments on the site.
db.php	(shared helper)	Central MySQL connection. Included by scripts/pages that may be run directly. Safe to include twice — it only connects if a connection does not already exist.
add_comment.php	Logged-in users	Receives the comment form, validates it, inserts the comment (prepared statement) into comment_text, then returns to the article.
admin_action.php	Admins only	The single handler for all admin buttons: publish / unpublish / delete a post, delete a comment, delete a user. Re-checks the admin role, uses prepared statements, and blocks an admin from deleting their own account.
user_content_action.php	Logged-in users	Lets a member delete their OWN post or comment. Every query includes "AND user_id = you", so members can never delete someone else's content.
update_profile.php	Logged-in users	Saves profile changes from the My Profile form: verifies the current password, checks the new email is not taken, and (optionally) hashes a new password.
assets/beautify.css	(shared style)	One stylesheet linked into every page that gives the whole site its premium look: self-hosted fonts, refined colours/shadows/buttons, and the scroll-animation styles. Loaded after each page's own CSS so it can refine it.
assets/beautify.js	(shared script)	Adds the subtle fade/slide-in as you scroll (using IntersectionObserver). Content stays fully visible even if this file fails.
assets/fonts/ , assets/img/ , favicon.svg	(assets)	Self-hosted Inter + Playfair Display fonts (works offline), local article cover photos, and the browser-tab icon.
seed_users.php , assets/dev/reseed_articles.php	(dev only)	Helper scripts: seed_users adds demo members (password 123, role reader); reseed_articles fills the site with the 45 realistic articles + sample comments. Delete both (and the backup_pre_beautify/ folder) before submission.

Note: the five helper files above (db.php, add_comment.php, admin_action.php, user_content_action.php, update_profile.php) complete the features the site links to — commenting, admin approval/deletion, deleting your own content, and profile editing. Every one of these flows has been tested end-to-end and works.

8. Key Workflows (Step By Step, In Simple Words)

8.1 Registration

- User fills username, first/last name, email, password + confirm password.
- PHP validates: nothing empty, passwords match, password at least 6 characters, email format valid (filter_var).
- A **prepared statement** checks the username/email is not already taken.
- The password is hashed with **password_hash()** (bcrypt) — the real password is never stored.

- The user is inserted with the default role **reader** (the first value of the role ENUM), and can now log in.

8.2 Login (and role-based redirect)

- PHP finds the user row by username.
- **password_verify()** compares the typed password against the stored hash (we never “decrypt” anything — hashing is one-way).
- On success we store **user_id, username, role** in \$_SESSION.
- If the role is admin/superadmin the browser is redirected to the admin dashboard; a normal user goes to the homepage.
- **Smart redirect:** if you were reading an article and logged in to comment, the site remembers that article's URL in the session and sends you straight back to it after login.
- Logout simply calls **session_destroy()** and returns to home.

8.3 Article Publishing — the heart of the CMS

User writes article -> saved in DB with status = 'pending' -> NOT visible on the site yet -> admin sees it in Manage Posts -> admin approves -> status becomes 'published' -> article now appears on the homepage and can be opened and commented on.

Why moderation? Because on a public news site you cannot let anyone publish instantly — the admin filters spam, abuse and fake news before it goes live. All public queries filter with `WHERE status = 'published'`, so pending posts are invisible everywhere on the public site.

8.4 Image handling in articles

- Option A — upload a file: it is moved into the **uploads/** folder with a timestamp added to the filename (time() + original name) so two files never clash.
- Option B — paste an image URL: the link is stored directly.
- If neither is given, the site shows a default placeholder image so the layout never breaks.

8.5 Comments

- Only logged-in users can comment (guests see a login prompt instead of the form).
- A comment stores post_id + user_id + text + timestamp, and appears under the article, newest first, with the commenter's username.

9. Security Features (Examiners Love This Section)

Threat / Concern	Our protection	How to explain it in one line
Stolen password database	password_hash() / password_verify() (bcrypt)	“We store only one-way hashes; even we cannot read a user's real password.”

Threat / Concern	Our protection	How to explain it in one line
SQL Injection	Prepared statements with <code>bind_param()</code> at the critical points (login, registration, duplicate checks); <code>real_escape_string()</code> on other inputs; (int) casting for numeric IDs like post id	"User input is never trusted; it is bound as data, not glued into SQL as code."
XSS (script injection into pages)	<code>htmlspecialchars()</code> whenever user-typed data is printed (titles, usernames, comments)	"Anything a user typed is escaped before display so a <code><script></code> tag becomes harmless text."
Unauthorized page access	Session + role checks at the top of every protected file	"Typing an admin URL directly just kicks you out — the page checks the session first."
Accidental privilege loss	<code>edit_user.php</code> blocks an admin from changing their own role	"A small business rule that prevents admins from locking themselves out."

If the examiner points out that some queries still concatenate strings: answer honestly — "Yes, we used prepared statements at the most security-critical points (authentication and registration) and escaping elsewhere; converting every remaining query to prepared statements is listed in our future improvements." Honesty + a plan scores better than pretending it is perfect.

10. User Interface & Design

The whole site shares one design system so every page looks like part of the same product. It is a blue news-portal theme built with hand-written CSS (no Bootstrap or other framework), which shows we understand CSS rather than relying on ready-made components.

- **One shared stylesheet:** every page links `assets/beautify.css`, loaded after the page's own styles so it can refine them. Because all pages reuse the same class names and CSS variables, this single file themes the entire site — change a colour once and it updates everywhere.
- **CSS variables (design tokens):** colours, shadows, radius and gradients are defined once in `:root` (e.g. `--primary`, `--shadow`) and reused with `var(--primary)` — the reason the look is consistent.
- **Self-hosted fonts:** Inter (body) and Playfair Display (headings) are stored in the project (`assets/fonts/`) instead of loaded from Google's servers, so the site looks identical **even with no internet**.
- **Subtle animations:** cards and sections gently fade/slide in as you scroll (`assets/beautify.js`, using the browser's `IntersectionObserver`). It respects the "reduced motion" accessibility setting, and content stays visible even if the script fails.
- **Responsive layout:** CSS media queries collapse the two-column layouts and hide the admin sidebar on small screens, so it works on phones as well as desktops.
- **Finishing touches:** a browser-tab favicon, gradient buttons and badges, hover lifts on cards, and a working newsletter acknowledgement.

Content: the site is filled with **45 realistic articles across 12 categories** (Technology, Business, Finance, Health, Science, Sports, Entertainment, Travel, Automotive, World, Food, Environment), each with a local cover image, so the homepage and the category filter show real, well-organised

content instead of placeholder text.

11. Expected Viva Questions — With Ready Answers

Q. What is a CMS?

A. A Content Management System — software that lets people create, edit, approve and publish website content through the website itself, without editing code. WordPress is the most famous example; ours is a custom-built news CMS.

Q. Why PHP and MySQL?

A. PHP is free, designed specifically for server-side web development, and runs on any hosting. MySQL is a reliable free relational database that pairs naturally with PHP through the built-in mysqli extension. XAMPP gives us Apache + PHP + MySQL in one package for development.

Q. Explain what happens when someone opens your website.

A. Apache receives the request and (via .htaccess) sends it to index.php. index.php starts the session, connects to the cms database, reads the ?page parameter and its switch-case loads the right PHP file. That file queries MySQL, mixes the data into HTML, and the finished page is returned to the browser.

Q. What is a session? How is it different from a cookie?

A. A session stores data about the current visitor on the SERVER (we store user_id, username, role). The browser only holds a small session-ID cookie. A cookie stores the data itself in the browser, which the user can see and edit — so sensitive login state belongs in the session.

Q. How are passwords stored?

A. Never in plain text. We call password_hash(), which uses the bcrypt algorithm with an automatic salt, and store only the hash. At login, password_verify() re-hashes the typed password and compares. Hashing is one-way — unlike encryption there is no key and no way to get the original back.

Q. What is SQL injection and how do you prevent it?

A. It is when an attacker types SQL code into a form field (like ' OR '1'='1) hoping it gets executed. We prevent it using prepared statements: the query template with ? placeholders is sent first, then values are bound with bind_param(), so input can only ever be data, never SQL commands. We also escape strings and cast IDs to integers.

Q. What is XSS and how do you prevent it?

A. Cross-Site Scripting: an attacker saves JavaScript inside a comment or title so it runs in other visitors' browsers. We print user data through htmlspecialchars(), which converts < and > into harmless text, so scripts cannot execute.

Q. Explain your database schema and the relationships.

A. Three tables. users (accounts, with a role column), posts (articles, each with a user_id foreign key to its author, plus category and status), comments (each with a post_id and a user_id). Relationships: one user to many posts, one post to many comments. We use INNER JOINS to show author/commenter names alongside content.

Q. What is a primary key and a foreign key?

A. A primary key uniquely identifies each row (id, auto-increment). A foreign key is a column that stores another table's primary key to link them — e.g. posts.user_id points to users.id, so we always know who wrote which article.

Q. Why does index.php use a switch-case? What pattern is that?

A. It is a simple router — the Front Controller pattern. One single entry point handles session and database setup once, then dispatches to the requested page. Benefits: no duplicated setup code, one place to control access and add new pages, and the site keeps a single consistent URL style.

Q. What does .htaccess do in your project?

A. It enables Apache URL rewriting: .html style URLs and the site root are internally redirected to index.php with the right ?page parameter. Users see clean URLs; the real file structure stays hidden.

Q. Difference between GET and POST? Where do you use each?

A. GET puts data in the URL — we use it for navigation and identifying things (?page=single_post&id=5). POST sends data hidden in the request body — we use it for forms: login, registration, writing articles, comments. Passwords must never appear in a URL.

Q. How does the post approval workflow work?

A. A user's new article is saved with status 'pending', which every public query filters out. The admin sees pending posts in Manage Posts and approves them, changing status to 'published' — only then does the article appear on the homepage. This is content moderation.

Q. How does image upload work?

A. The form uses multipart encoding; PHP receives the file in \$_FILES; we move it with move_uploaded_file() into the uploads/ folder, prefixing the filename with time() so names never collide; the resulting path is saved in posts.image_path. Alternatively the user can just paste an image URL.

Q. What is mysqli? Why not PDO?

A. Both are PHP database interfaces. mysqli is MySQL-specific and slightly simpler, which is why we chose it; PDO supports many databases and named parameters. Both support prepared statements — switching to PDO would be straightforward.

Q. What is role-based access control?

A. Permissions depend on the user's role, not on hiding links. We store role in the session and every protected page verifies it at the top, so even a direct URL to an admin page is blocked for non-admins.

Q. How would you improve the project in the future?

A. Convert all remaining queries to prepared statements; add pagination and a search box; password reset by email and email verification; likes/views counters; a rich-text editor for articles; CSRF tokens on forms; and eventually migrate to a framework like Laravel with a proper MVC structure and REST API. (The category filter is already working.)

Q. Is your site responsive?

A. Yes — the CSS uses media queries (e.g. at 960px and 640px) to collapse the two-column layout into one column, adapt the navbar, and hide the admin sidebar on small screens.

Q. How did you keep the design consistent across so many pages?

A. A shared design system. Colours, shadows and spacing are defined once as CSS variables, and one shared stylesheet (assets/beautify.css) is linked into every page and loaded after each page's own styles so it refines them. Because all pages reuse the same class names, changing the theme in one file updates the whole site.

Q. Why did you self-host the fonts instead of using Google Fonts?

A. So the site does not depend on an internet connection. The Inter and Playfair Display font files live inside the project (assets/fonts/), which means the design looks identical offline — important for a live demo where the connection might drop.

Q. How do the scroll animations work?

A. A small JavaScript file uses the browser's IntersectionObserver to detect when a card scrolls into view and adds a CSS class that fades and slides it in. It respects the user's reduced-motion accessibility setting, and the content is fully visible even if the script does not run, so it never hides anything.

Q. How does the category filter work?

A. The pills are generated from the real categories in the database. Clicking one reloads the homepage with `?category=Name`; the feed query then adds `AND category = ?` using a prepared statement (so it is injection-safe) and the matching pill is highlighted. 'All' clears the filter.

12. Demo Checklist For Monday (Whole Team, Please Read)

12.1 Before the viva

- Start XAMPP (Apache + MySQL) BEFORE entering; verify `http://localhost:8080/de6/index.php` opens.
- Keep two accounts ready and written down: the **admin** account and one **normal member**. The seeded members (mustufa, meet, yusuf, alice, bob, charlie, diana, ethan) all have password **123**.
- The site already has **45 articles across 12 categories** (4 kept pending for the approval demo), so the homepage looks full and professional.
- Delete the development-only files so the examiner cannot open them: the `seed_*.php` scripts, `assets/dev/reseed_articles.php`, and the `backup_pre_beautify/` folder.

12.2 Suggested demo order (tells the whole story)

1. Homepage as a guest — show the hero with live stats, the article cards, and the featured post. Mention the clean, custom design.
2. **Click a category pill** (e.g. Technology) — the feed filters to that category; a great chance to explain the prepared-statement filter.
3. Open an article and scroll — point out the comments and the subtle animations.

- 4. Register a fresh account live, then log in with it.
- 5. As that member: write a new article, point out the “waiting for admin verification” message; add a comment on an article.
- 6. Show the new article is NOT on the homepage yet (status pending).
- 7. Log out, log in as admin — land on the dashboard, show the live statistics.
- 8. Approve the pending article in Manage Posts, then show it live on the homepage.
- 9. Show Manage Users / edit a user's role to demonstrate admin power.

Status — everything works and is polished (tested end-to-end):

All features function: registration, login, writing articles, commenting, admin approve / unpublish / delete, deleting your own content, and profile editing. Earlier fixes (the missing helper files, a double-close database error, and an invalid seed role) are all resolved. The site has since been given a consistent premium design (shared stylesheet, self-hosted offline fonts, subtle animations, favicon) and filled with 45 real articles. **Only reminder:** start MySQL + Apache in XAMPP before the viva, and delete the development-only files (seed_*.php, assets/dev/, backup_pre_beautify/) before final submission.

12.3 What every team member must be able to answer alone

- The one-line project summary (Section 1) — word perfect.
- The request flow through index.php (Section 4).
- The three tables and how they connect (Section 5).
- How login security works: hashing + sessions + role checks (Sections 8 and 9).
- The pending-to-published approval workflow (Section 8.3).

13. The One-Minute Opening Pitch (Memorize This)

“Our final year project is a News Portal Content Management System built with PHP and MySQL on XAMPP. The whole site runs through a single router file, index.php, which manages the session, the database connection and page loading — the Front Controller pattern. There are three roles: guests read published news; registered users write articles and comment; admins moderate everything from a dashboard. The key feature is the publishing workflow: a user's article is saved as pending and only appears publicly after an admin approves it. The database has three related tables — users, posts and comments — joined by foreign keys. For security we hash passwords with bcrypt, use prepared statements against SQL injection, escape all output against XSS, and protect every page with session-based, role-based access control. The whole site shares one custom, responsive design system with self-hosted fonts so it looks consistent and works even offline.”

Good luck on Monday — you built this, you know it. Speak slowly, demo confidently, and if you don't know an answer, explain how you would find out.